Cartesian Path

Cartesian Path

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

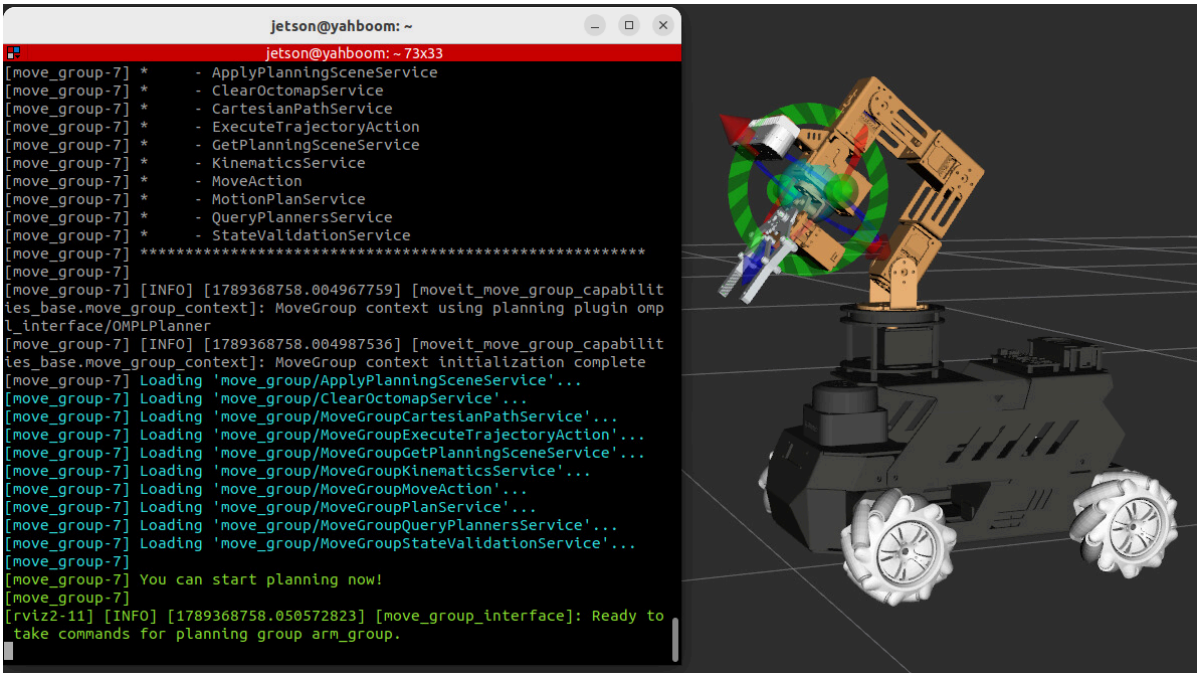
1. Course Content

This section explains how to use the functions in MoveIt2 to implement a Cartesian path. Cartesian path planning is an important feature in robot arm control. It allows the user to specify the motion trajectory of the end effector in Cartesian space (i.e., 3D space), rather than directly specifying the motion in joint space. MoveIt provides a powerful Cartesian path planning capability.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



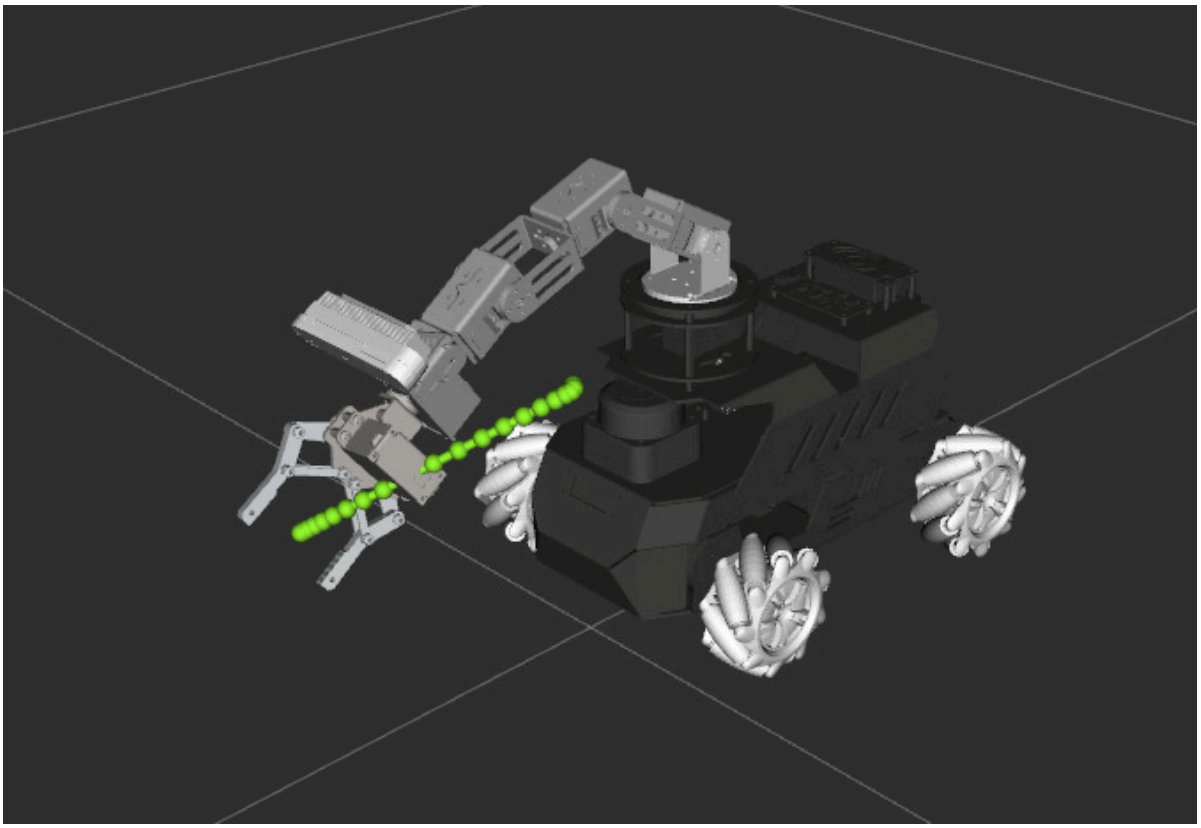
3. Case Demonstration

```
ros2 run moveit_demo cartesian_path --ros-args -p start_x:=0.1 -p start_y:=0.10 -
p start_z:=0.23 -p end_x:=0.2 -p end_y:=0.004 -p end_z:=0.15
```

> **Tip**

> - The start position and end position in the command here can be specified by yourself through parameters.

After running, the robot arm end will move from the specified start position to the pose set by the end position, and the green Cartesian path points will be displayed, as shown in the figure below:



4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/cartesian_path.cpp
```

initialize function description:

- `declare_parameter("start_x/y/z", ...)` / `"end_x/y/z"`: Declares the start and end position parameters of the Cartesian path (unit: meters), with the coordinate frame being measure.
- `eef_step`: End-effector step distance (default 0.01 m), i.e., the maximum distance between adjacent interpolation points in the Cartesian path; the smaller the value, the smoother the path.
- `jump_threshold`: Joint jump threshold (default 0.0); setting it to 0 disables joint jump detection, while a non-zero value is used to limit the maximum change in joint angle between adjacent path points, preventing sudden large movements of the robot arm.
- `maxtries`: Maximum number of retries for Cartesian path computation (default 100).
- `planning_time`: Maximum time for a single planning attempt (default 10.0 seconds).
- `setEndEffectorLink("gripper_tcp")`: Specifies the end effector link as `gripper_tcp`.
- `setPoseReferenceFrame("measure")`: Sets the pose reference coordinate frame to measure.

- `getJointModelGroup("arm_group")`: Gets the arm_group planning group, used to reference the joint model during visualization.
- `MoveItVisualTools(shared_from_this(), "measure", "plan_point", ...)`: Creates an RViz visualization tool, with the reference frame being measure and the published topic being plan_point.

Phase 1 — Move to the start position:

- `setPositionTarget(sx, sy, sz)`: Sets the start position as the target, specifying only the position and not the orientation.
- `plan(start_plan)` / `execute(start_plan)`: Plans and executes the move to the start position; if it fails, the node is shut down.
- `points.back()`: Takes the last point of the trajectory and prints the joint angles after reaching the start position.

Phase 2 — Get the current pose:

- `getCurrentPose("gripper_tcp")`: Gets the current pose of the gripper end, used to extract the orientation information, retrying up to 20 times.
- On failure, it falls back to building a default pose (orientation w=1.0) using the start position.

Phase 3 — Build the Cartesian waypoints:

- `waypoints`: The waypoint list, containing two poses: the start position and the end position.
- `start_pose.orientation = current_pose.pose.orientation`: Both the start and end preserve the current orientation, changing only the position.

Phase 4 — Compute and execute the Cartesian path:

- `computeCartesianPath(waypoints, eef_step, jump_threshold, trajectory_msg)`: Computes the Cartesian path based on the waypoint list, step distance, and jump threshold, returning the completion ratio fraction (0.0~1.0).
- `fraction > 0.0`: Determines whether the path computation succeeded; if so, visualize and execute it.
- `publishTrajectoryLine(...)`: Draws the Cartesian trajectory line of the end effector in cyan-green in RViz.
- `cartesian_plan.trajectory_ = trajectory_msg`: Assigns the computed trajectory to the Plan object.
- `execute(cartesian_plan)`: Executes the Cartesian path, driving the robot arm end to move in a straight line from the start position to the end position.
- `attempts >= maxtries`: Reports an error if it still fails after exceeding the maximum number of retries.

```
class CartesianPathPlanning : public rclcpp::Node
{
public:
  CartesianPathPlanning()
    : Node("cartesian_path_planning")
  {
  }
```

```
  void initialize()
  {
    // ===== 1. Parameters: start and end positions (xyz only, no orientation)
=====
    this->declare_parameter("start_x", -0.069);
    this->declare_parameter("start_y", 0.004);
    this->declare_parameter("start_z", 0.15);
    this->declare_parameter("end_x", 0.18);
    this->declare_parameter("end_y", 0.004);
    this->declare_parameter("end_z", 0.15);
    this->declare_parameter("eef_step", 0.01);
    this->declare_parameter("jump_threshold", 0.0);
    this->declare_parameter("maxtries", 100);
    this->declare_parameter("planning_time", 10.0);

    double sx = this->get_parameter("start_x").as_double();
    double sy = this->get_parameter("start_y").as_double();
    double sz = this->get_parameter("start_z").as_double();
    double ex = this->get_parameter("end_x").as_double();
    double ey = this->get_parameter("end_y").as_double();
    double ez = this->get_parameter("end_z").as_double();
    double eef_step = this->get_parameter("eef_step").as_double();
    double jump_threshold = this->get_parameter("jump_threshold").as_double();
    int maxtries = this->get_parameter("maxtries").as_int();
    double planning_time = this->get_parameter("planning_time").as_double();

    RCLCPP_INFO(this->get_logger(),
      "Start position (measure frame): [%.4f, %.4f, %.4f]", sx, sy, sz);
    RCLCPP_INFO(this->get_logger(),
      "End position (measure frame): [%.4f, %.4f, %.4f]", ex, ey, ez);

    // ===== 2. MoveGroupInterface =====
    move_group_interface_ =
      std::make_shared<moveit::planning_interface::MoveGroupInterface>(
        shared_from_this(), "arm_group");

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(planning_time);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);
    move_group_interface_->setEndEffectorLink("gripper_tcp");
    move_group_interface_->setPoseReferenceFrame("measure");
    move_group_interface_->setGoalJointTolerance(0.01);
    move_group_interface_->setGoalPositionTolerance(0.005);

    // ===== 2.5 Visual tools =====
    const moveit::core::JointModelGroup* joint_model_group =
      move_group_interface_->getRobotModel()->getJointModelGroup("arm_group");
    visual_tools_ = std::make_shared<moveit_visual_tools::MoveItVisualTools>(
      shared_from_this(), "measure", "plan_point",
      move_group_interface_->getRobotModel());
    visual_tools_->deleteAllMarkers();
    visual_tools_->loadRemoteControl();

    // ===== 3. Phase 1: Move to start position via position-only planning =====
    move_group_interface_->setPositionTarget(sx, sy, sz);
```

```
    RCLCPP_INFO(this->get_logger(), "Planning move to start position...");
    moveit::planning_interface::MoveGroupInterface::Plan start_plan;
    if (move_group_interface_->plan(start_plan) !=
        moveit::core::MoveItErrorCode::SUCCESS)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Failed to plan move to start position! Target may be out of workspace or
cause collision.");
      rclcpp::shutdown();
      return;
    }

    RCLCPP_INFO(this->get_logger(), "Executing move to start position...");
    move_group_interface_->execute(start_plan);

    // Print joint angles after reaching start
    {
      const auto& points = start_plan.trajectory_.joint_trajectory.points;
      if (!points.empty())
      {
        const auto& names = start_plan.trajectory_.joint_trajectory.joint_names;
        const auto& last = points.back();
        RCLCPP_INFO(this->get_logger(), "Reached start position, joint angles:");
        for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
        {
          RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
            names[i].c_str(), last.positions[i],
            last.positions[i] * 180.0 / M_PI);
        }
      }
    }

    // ===== 4. Get current pose for orientation (retry if needed) =====
    geometry_msgs::msg::PoseStamped current_pose;
    bool pose_ok = false;
    for (int retry = 0; retry < 20 && rclcpp::ok(); retry++)
    {
      try
      {
        current_pose = move_group_interface_->getCurrentPose("gripper_tcp");
        pose_ok = true;
        RCLCPP_INFO(this->get_logger(), "Current pose obtained after %d retries",
retry);
        break;
      }
      catch (...)
      {
        RCLCPP_WARN(this->get_logger(),
          "Waiting for robot state (retry %d/20)...", retry + 1);
        rclcpp::sleep_for(std::chrono::milliseconds(200));
      }
    }

    if (!pose_ok)
    {
```

```
      RCLCPP_ERROR(this->get_logger(),
        "Failed to get current pose after retries. Falling back to start position
IK result.");
      // Build pose from start plan's IK result as fallback
      current_pose.header.frame_id = "measure";
      current_pose.pose.position.x = sx;
      current_pose.pose.position.y = sy;
      current_pose.pose.position.z = sz;
      current_pose.pose.orientation.w = 1.0;
    }

    // ===== 5. Phase 2: Build Cartesian waypoints (position only, preserve
orientation) =====
    std::vector<geometry_msgs::msg::Pose> waypoints;

    geometry_msgs::msg::Pose start_pose;
    start_pose.position.x = sx;
    start_pose.position.y = sy;
    start_pose.position.z = sz;
    start_pose.orientation = current_pose.pose.orientation;
    waypoints.push_back(start_pose);

    geometry_msgs::msg::Pose end_pose;
    end_pose.position.x = ex;
    end_pose.position.y = ey;
    end_pose.position.z = ez;
    end_pose.orientation = current_pose.pose.orientation;
    waypoints.push_back(end_pose);

    // ===== 6. Compute Cartesian path =====
    moveit_msgs::msg::RobotTrajectory trajectory_msg;
    int attempts = 0;
    double fraction = 0.0;

    while (attempts < maxtries)
    {
      fraction = move_group_interface_->computeCartesianPath(
        waypoints, eef_step, jump_threshold, trajectory_msg);
      if (fraction > 0.0)
      {
        RCLCPP_INFO(this->get_logger(),
          "Cartesian path planned successfully (%.2f%% achieved)", fraction *
100.0);

        // Visualize trajectory in RViz
        visual_tools_->publishTrajectoryLine(trajectory_msg,
          move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
          joint_model_group, rviz_visual_tools::LIME_GREEN);
        visual_tools_->trigger();

        moveit::planning_interface::MoveGroupInterface::Plan cartesian_plan;
        cartesian_plan.trajectory_ = trajectory_msg;
        RCLCPP_INFO(this->get_logger(), "Executing Cartesian path...");
        move_group_interface_->execute(cartesian_plan);

        // Print final joint angles
```

```
        const auto& points = cartesian_plan.trajectory_.joint_trajectory.points;
        if (!points.empty())
        {
          const auto& names =
cartesian_plan.trajectory_.joint_trajectory.joint_names;
          const auto& last = points.back();
          RCLCPP_INFO(this->get_logger(), "Execution complete, final joint
angles:");
          for (size_t i = 0; i < names.size() && i < last.positions.size(); i++)
          {
            RCLCPP_INFO(this->get_logger(), "  %s = %.4f rad (%.2f deg)",
              names[i].c_str(), last.positions[i],
              last.positions[i] * 180.0 / M_PI);
          }
        }
        break;
      }
      else
      {
        RCLCPP_ERROR(this->get_logger(),
          "Failed to compute Cartesian path (attempt %d/%d)", attempts + 1,
maxtries);
        attempts++;
      }
    }

    if (attempts >= maxtries)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Cartesian path planning failed after %d attempts!", maxtries);
    }

    rclcpp::shutdown();
  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};
```